OpenCV Geometric Transforms

1. Experiment Objective

Master image geometric transforms: scaling, translation, cropping, mirror flipping, and other basic spatial transform operations.

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
Affine transform	The combination of linear transforms (rotation, scaling, translation) with translation, preserving the parallel relationship of lines in the image
Translation matrix	A 2×3 transform matrix of the form `[[1,0,tx],[0,1,ty]]` that moves pixels along the x and y axes
Bilinear interpolation	The interpolation algorithm used by default in `cv2.resize()`, computing new pixel values from the weighted average of the surrounding 2×2 pixels
ROI	Region of Interest; a sub-region cropped from the original image via array slicing

2. Technical Architecture

This experiment is a pure offline Python script. The core of geometric transforms is **coordinate mapping** — for each pixel coordinate in the output image, the transform matrix is used to compute its corresponding coordinate in the input image, which is then sampled and filled.

Data flow: local image → `cv2.imread()` → build the transform matrix (or slice directly) → `cv2.warpAffine()` / `cv2.resize()` → side-by-side comparison display in two windows.

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☑ Supported
No-PTZ version	☑ Supported
Required peripherals	None (local image processing, no need to start the car)

2. Startup Method

Mode 1: Translation

```
python3
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_geometric_transform
ation/opencv_image_panning.py
```

Mode 2: Cropping

```
python3
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_geometric_transform
ation/opencv_image_cropping.py
```

Mode 3: Mirror flipping

```
python3
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_geometric_transform
ation/opencv_image_mirroring.py
```

Mode 4: Scaling (zoom out + zoom in)

```
python3
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_geometric_transform
ation/opencv_zoom_out_in.py
```

3. Operation Instructions

After each script runs, a window pops up displaying the original image and the transformed result side by side:

- `opencv_image_panning.py`: original image + result translated 30 pixels down and to the right
- `opencv_image_cropping.py`: original image + cropped ROI region (y=0~100, x=100~200)
- `opencv_image_mirroring.py`: original image + vertically mirrored flip (vertically stacked)
- `opencv_zoom_out_in.py`: original image + zoomed out 50% + zoomed in 200%

Press `q` to exit.

4. Stop Method

Press `q` to close all OpenCV windows and exit the program.

4. Experiment Phenomena

- Understand geometric operations such as `cv2.warpAffine()`, `cv2.resize()`, and NumPy slicing
- Windows display the original image and the transformed result side by side

Original image: The input image for geometric transform operations; all transforms are compared and displayed against this image.

Translation: The image is moved by the specified number of pixels along the x/y axes, with black fill areas appearing on the right and bottom.

Cropping: Crops a local region of the image via NumPy array slicing `img[y1:y2, x1:x2]`.

Mirroring: Achieves vertical flipping by copying pixels row by row, producing a top-bottom symmetric effect.

Zoom out: Uses `cv2.resize()` to scale the image dimensions down proportionally; the pixel count decreases but the main content is preserved.

Zoom in: Uses `cv2.resize()` to scale the image dimensions up proportionally, with the interpolation algorithm filling in the new pixels.

5. Program Analysis

> Source code path:

> - Script directory: `~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_geometric_transformation/`
> - Test image: `~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/images/yahboom.png`

1. Core Logic Analysis

The four geometric transforms share the same execution framework; they differ in the transform parameters and API calls.

Affine translation (opencv_image_panning.py)

```
# Source file:
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_geometric_transform
ation/opencv_image_panning.py
import cv2
import numpy as np

if __name__ == '__main__':
    img =
cv2.imread('~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/images/yahboom
.png', cv2.IMREAD_UNCHANGED)
    if img is None:
        print("Image read failed, please check that the images/yahboom.png path
is correct")
        exit()

    imgInfo = img.shape
    height = imgInfo[0]
    width = imgInfo[1]

    # Define the translation matrix: move 30 pixels to the right and 30 pixels
down
    matShift = np.float32([[1, 0, 30], [0, 1, 30]])

    dst = cv2.warpAffine(img, matShift, (width, height))

    while True:
        cv2.imshow("yahboom", img)
        cv2.imshow('yahboom_panned', dst)
        action = cv2.waitKey(10) & 0xFF
        if action == ord('q') or action == 113:
            break

    cv2.destroyAllWindows()
```

Key logic notes:

- `np.float32([[1,0,tx],[0,1,ty]])` builds a 2×3 affine translation matrix; `tx` and `ty` are the displacement (in pixels) along the x and y directions respectively
- `cv2.warpAffine(src, M, dsize)` applies an affine transform to the image; the third parameter specifies the output canvas size
- After translation, the areas vacated by the original pixels are filled with black (value 0)

Image cropping (opencv_image_cropping.py)

```
# Source file:
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_geometric_transform
ation/opencv_image_cropping.py
import cv2

if __name__ == '__main__':
    img =
cv2.imread('/home/l_jh/microros_ws/src/microros_v2_opencv_base_pkg/scripts/image
s/yahboom.png', cv2.IMREAD_UNCHANGED)
    if img is None:
        print("Image read failed, please check that the images/yahboom.png path
is correct")
        exit()

    dst = img[0:100, 100:200]

    while True:
        cv2.imshow("yahboom", img)
        cv2.imshow('yahboom_cropped', dst)
        action = cv2.waitKey(10) & 0xFF
        if action == ord('q') or action == 113:
            break

    cv2.destroyAllWindows()
```

Key logic notes:

- `img[y1:y2, x1:x2]` uses NumPy array slicing to directly extract a pixel subset without calling any OpenCV API
- This is the most efficient of all geometric transforms — a zero-copy view that only changes the access boundaries
- Slice intervals are left-closed and right-open: `0:100` means indices 0~99

Scaling (opencv_zoom_out_in.py)

```
# Source file:
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_geometric_transform
ation/opencv_zoom_out_in.py
import cv2

if __name__ == '__main__':
    img =
cv2.imread('~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/images/yahboom
.png', cv2.IMREAD_UNCHANGED)
    if img is None:
        print("Image read failed, please check that the images/yahboom.png path
is correct")
        exit()

    print(img.shape)

    x, y = img.shape[0:2]

    img_zoom_out = cv2.resize(img, (int(y / 2), int(x / 2)))
    img_zoom_in = cv2.resize(img, (y * 2, x * 2))

    while True:
```

```
        cv2.imshow("yahboom", img)
        cv2.imshow('yahboom_zoom_out', img_zoom_out)
        cv2.imshow('yahboom_zoom_in', img_zoom_in)
        action = cv2.waitKey(10) & 0xFF
        if action == ord('q') or action == 113:
            break

    cv2.destroyAllWindows()
```

Key logic notes:

- `cv2.resize(src, dsize)` scales the image to the target size; `dsize` is in `(width, height)` format
- Note that `img.shape` returns `(height, width)`, while the second parameter of `cv2.resize` expects `(width, height)` — the two orders are reversed
- Bilinear interpolation (`INTER_LINEAR`) is used by default; zooming out and zooming in share the same API

2. Key Parameters

Parameter	Type	Default	Description
`matShift`'s tx/ty	float32	30, 30	Translation amount (pixels); positive values move right/down
`warpAffine`'s dsize	tuple	(width, height)	Output canvas size; if the same size as the original image, the overflowing parts are cropped
`resize`'s dsize	tuple	—	Target size `(width, height)`; note the order difference from `shape`
NumPy slice range	slice	—	`img[y1:y2, x1:x2]`, a left-closed right-open interval

3. Node Notes

This experiment is a pure Python script and does not use ROS2 nodes. The script uses `if __name__ == '__main__':` as its entry point and runs independently in a desktop environment.

6. Common Problems

Problem	Cause	Solution
The transformed image goes out of bounds	Output size not adjusted	Enlarge the output canvas size or reduce the translation amount
`resize` size error	width/height order reversed	`cv2.resize` takes `(width, height)`, while `img.shape` is `(height, width)`
The cropped region is empty	Slice index out of bounds	Make sure the slice range is within the original image size: y2 ≤ height, x2 ≤ width